La capa de Transporte

Transporte orientado a conexión: TCP

Tema 3: La capa de Transporte

3.1 La capa de Transporte y sus servicios
3.2 Multiplexacción y demultiplexación
3.3 Transporte sin conexión: UDP
3.4 Principios de transferencia de datos fiable
3.5 Transporte orientado a conexión: TCP
- 3.5.1 La conexión TCP
- 3.5.2 Estructura del segmento TCP
- 3.5.3 Estimación del tiempo de ida y vuelta y fin de temporización
- 3.5.4 Transferencia de datos fiable
- 3.5.5 Control de flujo
- 3.5.6 Gestión de la conexión TCP

3.6 Principios de control de congestión
3.7 Mecanismo de control de congestión de TCP

TCP: Estimación del round-trip time y fin de temporización

- ¿Cómo TCP se recupera ante la pérdida de segmentos?
  - Con Temporizadores y procedimientos ARQ.
- Gestión de Temporizadores en TCP (RFC 2988).
  - En base al valor del Round-Trip Time (RTT) ➔ *Timeout* > RTT
    - RTT varía para cada segmento ➔ Se calculan RTTs estimados.
  - *Timeout* = Intervalo_Fin_Temporización = $RTT_{estimado} + 4 * RTT_{desv}$

TCP: Estimación del RTT y fin de temporización

- $RTT_{muestra}$ y $RTT_{estimado}$ para una conexión TCP entre EE.UU. y Francia

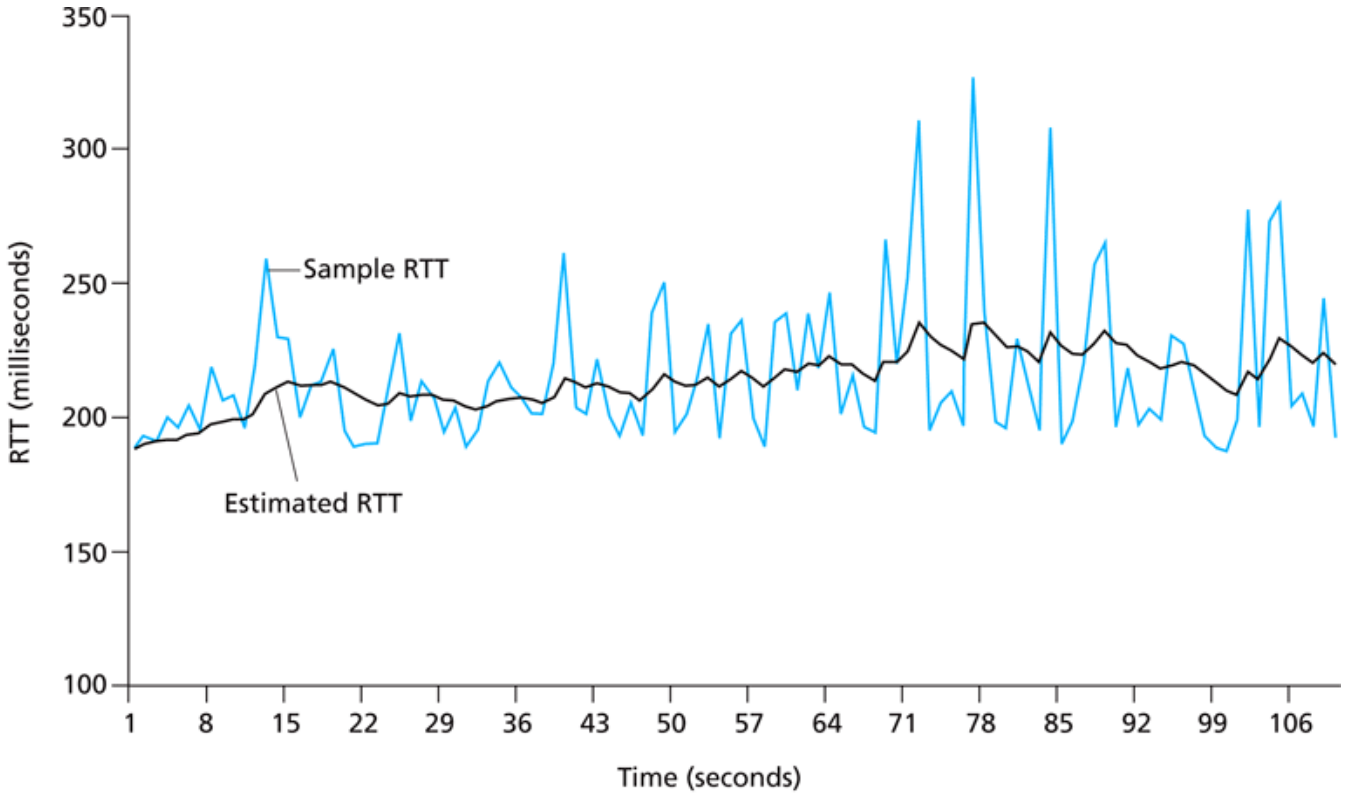


Figure 3.32 ♦ RTT samples and RTT estimates

$RTT_{muestra}$: tiempo que transcurre desde que se pasa el segmento a la capa IP (se envía) hasta que se recibe su reconocimiento.

$\boldsymbol{RTT_{estimado}}$: valor promedio de los valores RTTmuestra.

Nuevo valor $\boldsymbol{RTT_{muestra}}$ cada **RTT** segundos aproximadamente

Tema 3: La capa de Transporte

3.1 La capa de Transporte y sus servicios
3.2 Multiplexacción y demultiplexación
3.3 Transporte sin conexión: UDP
3.4 Principios de transferencia de datos fiable
3.5 Transporte orientado a conexión: TCP
- 3.5.1 La conexión TCP
- 3.5.2 Estructura del segmento TCP
- 3.5.3 Estimación del tiempo de ida y vuelta y fin de temporización
- 3.5.4 Transferencia de datos fiable
- 3.5.5 Control de flujo
- 3.5.6 Gestión de la conexión TCP

3.6 Principios de control de congestión
3.7 Mecanismo de control de congestión de TCP

TCP: Transferencia de datos fiable

- Gestión de Temporizadores
  - Carga de trabajo que puede ser considerable.
- En consecuencia, para TCP la RFC 2988 recomienda:
  - Un único temporizador de retransmisión, independientemente del Nº de segmentos pendientes de ser confirmados.

- Sucesos importantes en el Emisor TCP, relativos a la transmisión y retransmisión de datos, son:
  - Datos recibidos desde la capa de Aplicación.
  - Fin de la temporización (*Timeout*).
  - Recepción de un ACK.

TCP: Sucesos importantes en el Emisor TCP

- Datos recibidos desde la capa de Aplicación.
  - TCP lo encapsula en un segmento y lo pasa a la capa IP.
  - Se inicia el Temporizador en el caso de no estar iniciado previamente.
- Fin de temporización (timeout) .
  - TCP retransmite el segmento asociado a dicha temporización (segmento más antiguo, caso que explica el Kurose).*
  - Se reinicia el Temporizador.
- Recepción de un segmento con ACK válido.
  - Compara el valor ACK "y" con la variable "BaseEmision".
    - BaseEmision = Nº de secuencia más antiguo pendiente de ACK.
  - Si "y" > BaseEmision ➔ El ACK confirma uno o más segmentos anteriores **NO** reconocidos
    - Se actualiza el valor de la variable BaseEmision.
    - Si hay segmentos aún **NO** reconocidos ➔ Se reinicia el Temporizador.

TCP: Transferencia de datos fiable

Escenario posible

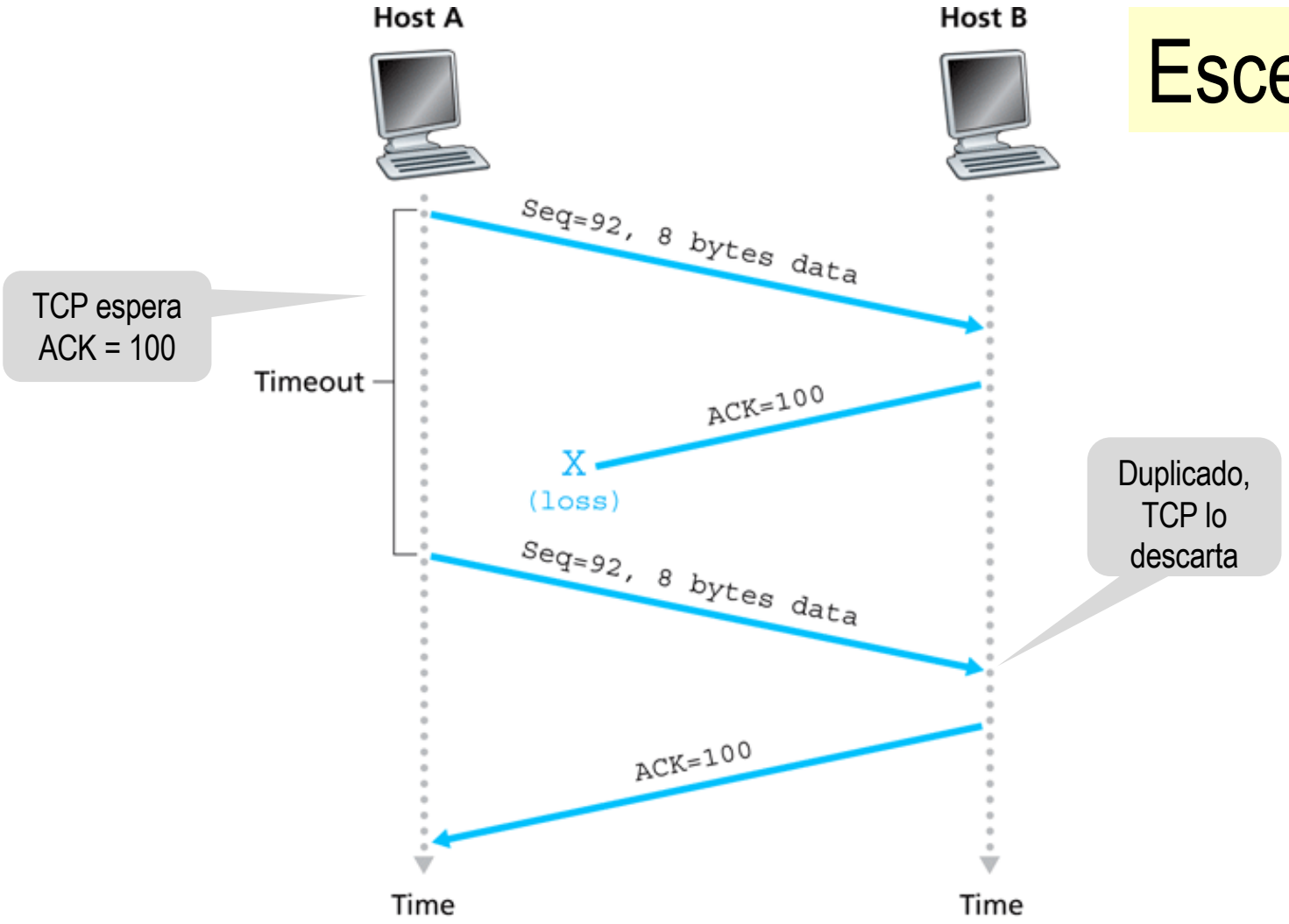


Figure 3.34 ♦ Retransmission due to a lost acknowledgment

TCP: Transferencia de datos fiable

Escenario posible

Caso en que sólo se retransmite el segmento más antiguo por confirmar al vencer el *Timeout*

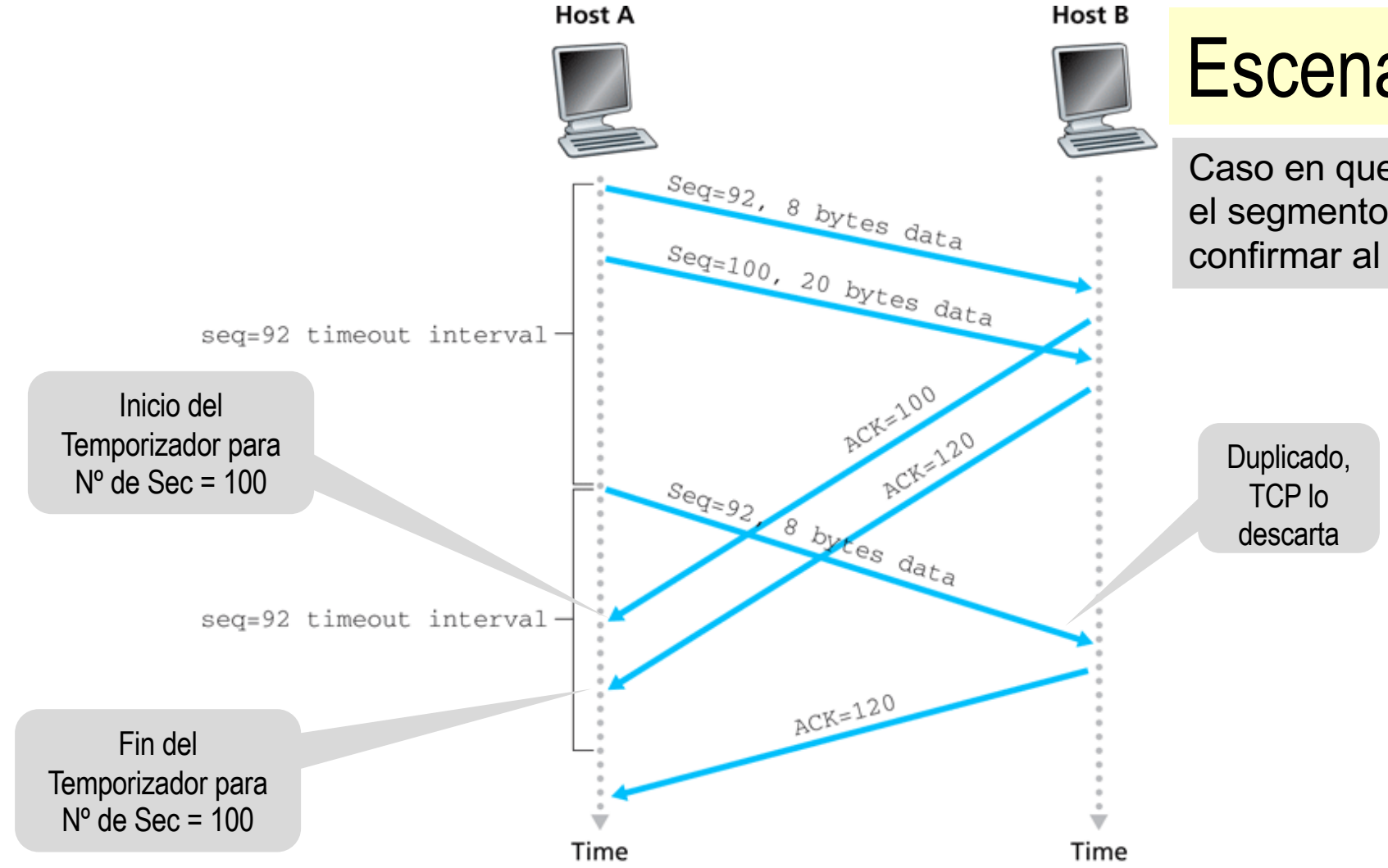


Figure 3.35 ♦ Segment 100 not retransmitted

TCP: Transferencia de datos fiable

Escenario posible

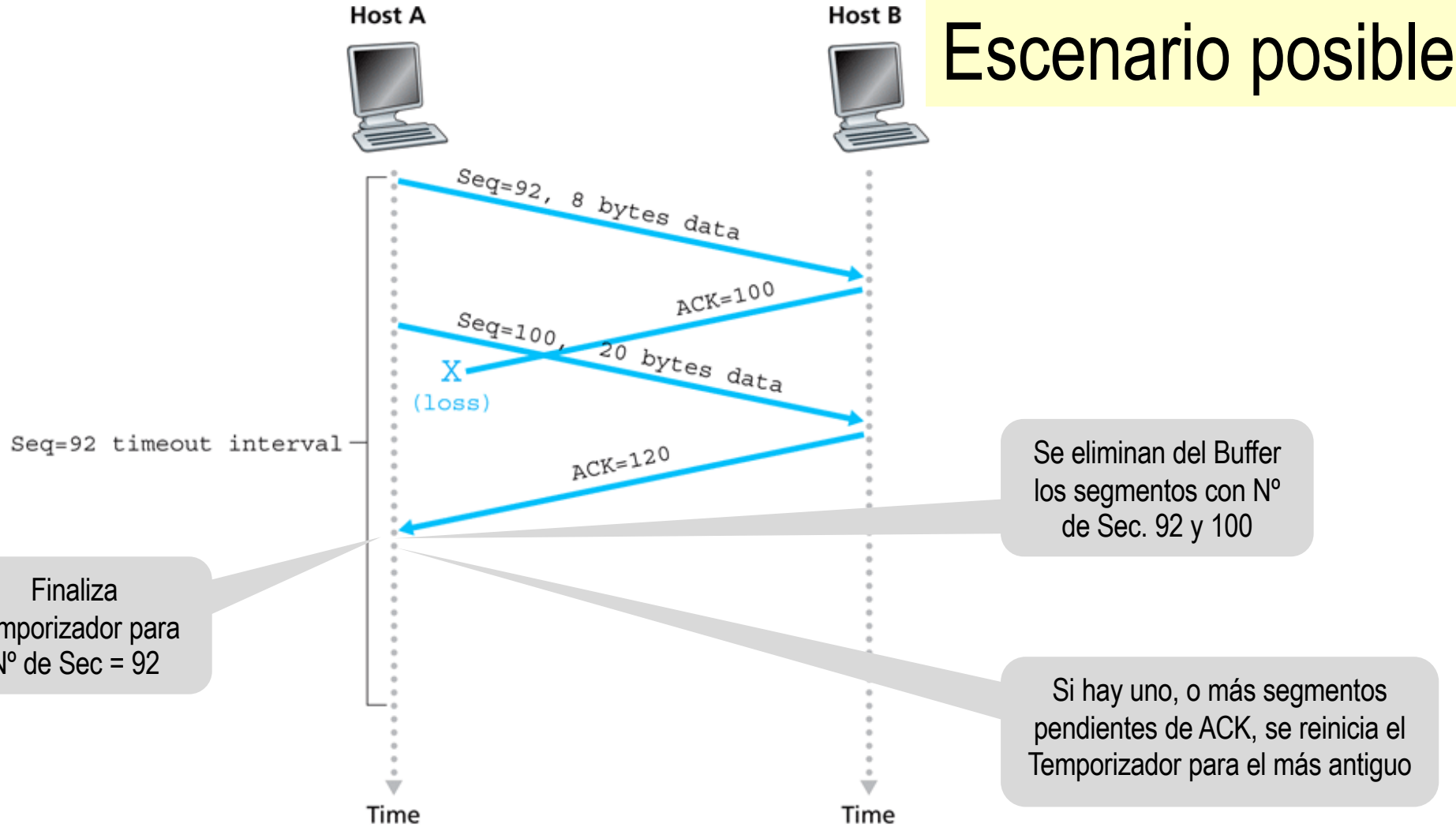


Figure 3.36 ♦ A cumulative acknowledgment avoids retransmission of the first segment.

TCP: Gestión del Timeout

- Cada vez que TCP retransmite, redefine el *Timeout*.
  - En este caso se obtiene el *Timeout como 2 veces el Timeout anterior*
  - Los *Timeouts* crecen exponencialmente después de cada retransmisión.
  - Este hecho permite tener **un control de congestión limitado**.
    - Se trata de una actuación "consciente" de TCP frente a congestiones en la red, aumenta los intervalos de retransmisión para evitar el empeoramiento de la congestión.
- En los otros dos casos, es decir, cuando se reciba un segmento de "datos de Aplicación" y/o de "ACK":
  - El nuevo *Timeout* se calcula como:
    - *Timeout* = $RTT_{estimado} + 4 * RTT_{desv}$

TCP: Retransmisión Rápida

- Es una solución frente a problemas derivados de las retransmisiones por vencimiento del *Timeout.*
  - El *Timeout* puede ser grande ➔ Se retarda la retransmisión ➔ Aumenta el retardo entre los Sistemas Finales.
- Solución práctica ➔ ACKs duplicados (NAK implícito).
  - ACK duplicados respecto al último octeto de datos recibido en secuencia.
  - Permiten al Emisor detectar, a menudo, la pérdida de paquetes antes de que caduque el *Timeout* ➔ Se reduce tiempo de espera para retransmitir.
  - Si el Emisor recibe tres (3) ACKs duplicados* ➔ Asume perdido el segmento que sigue al segmento confirmado tres veces ➔ Emisor realiza una retransmisión "rápida" antes que caduque el *Timeout* del segmento.

*: Criterio seguido en la mayoría de las implementaciones de TCP.

TCP: Ejemplo de Retransmisión Rápida

- Segundo segmento, con Nº de secuencia 100, se retransmite antes que caduque su Temporizador

Se detiene el Temporizador para el Nº de Sec. 92, y se inicia para el Nº de Sec. 100

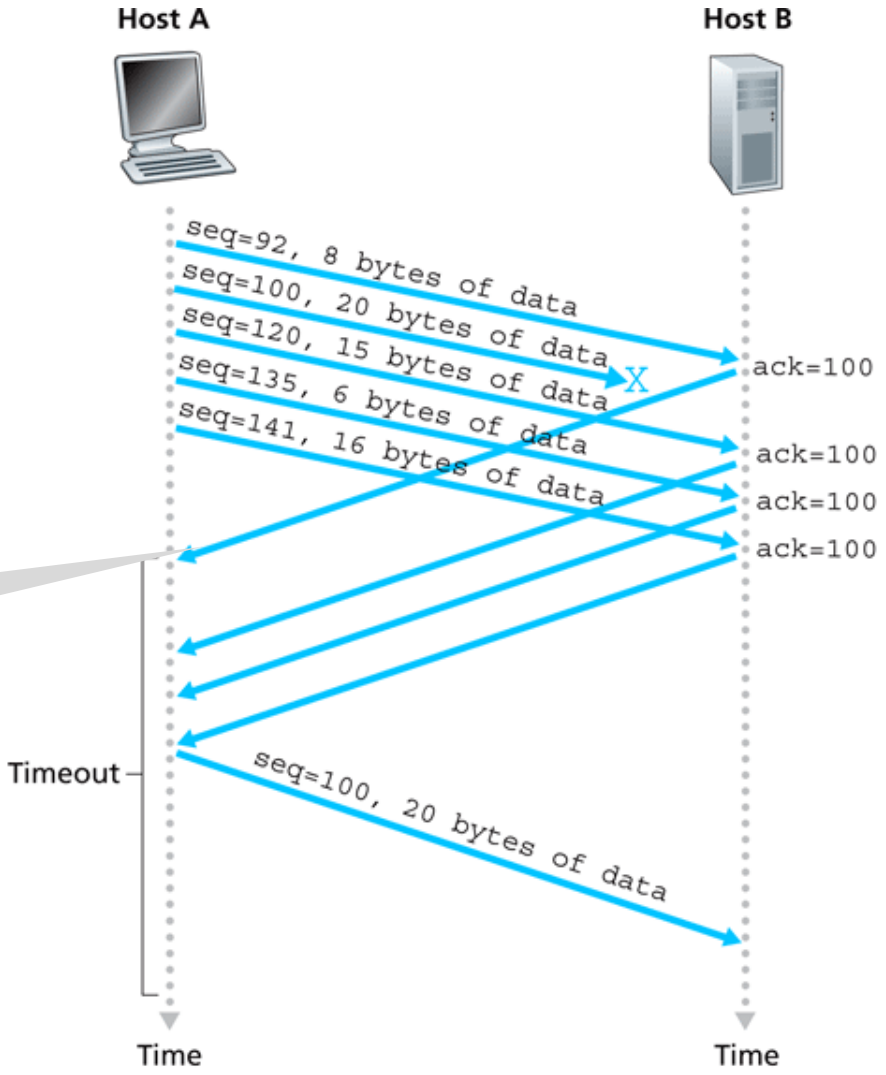


Figure 3.37 ♦ Fast retransmit: retransmitting the missing segment before the segment's timer expires.

TCP: Política de generación de ACKs por el **Receptor TCP**.

- Recomendación según RFCs 1122 y 2581.
- NO descarta segmentos fuera de secuencia. Asentimientos acumulativos.

	Suceso	Acción del Receptor TCP
Fila 1	Se recibe segmento con Nº de secuencia esperado. Nºs de secuencia anteriores ya reconocidos. No hay otro segmento en orden esperando transmisión de un ACK.	**ACK retardado*.** Se espera 500 mseg la llegada de otro segmento en secuencia. Si no llega, se envía ACK.
Fila 2	Se recibe segmento con Nº de secuencia esperado. Hay otro segmento en orden esperando transmisión de ACK.	**ACK único acumulativo**. De inmediato se reconocen ambos segmentos ordenados.
Fila 3	Se recibe segmento fuera de secuencia, con Nº mayor que el esperado. Se detecta un "hueco".	**ACK duplicado**. Se envía de inmediato ACK con Nº de secuencia del siguiente octeto esperado (límite inferior del "hueco").
Fila 4	Se recibe segmento que completa parcial o totalmente "hueco" en los datos recibidos.	**ACK inmediato**. Se envía ACK de inmediato si el segmento comienza en el límite inferior del "hueco". De lo contrario, ACK duplicado de inmediato.

Supuesto*: El host receptor, su transmisor TCP, no tiene segmentos de datos que enviar, de lo contrario envía los ACKs en dichos segmentos sin más, sin regirse por lo indicado en la 1ª fila de la tabla.

TCP: Recuperación de errores

- ¿Retroceder N (GBN) o Repetición Selectiva (SR)?
  - TCP utiliza confirmaciones acumulativas.
  - Segmentos bien recibidos, pero fuera de secuencia, **NO** se reconocen de manera individual por el **Receptor TCP** (ACKs duplicados).
    - Emisor sólo necesita mantener las variables "**BaseEmision**" y "**SigNumSec**"
    - Muchas implementaciones TCP almacenan en el *buffer* del Receptor segmentos fuera de secuencia.
  - TCP provee retransmisión selectiva (versión Reno de TCP).

Por tanto, el procedimiento TCP para recuperación de errores puede ser considerado una solución HÍBRIDA entre GBN y SR

Tema 3: La capa de Transporte

3.1 La capa de Transporte y sus servicios
3.2 Multiplexacción y demultiplexación
3.3 Transporte sin conexión: UDP
3.4 Principios de transferencia de datos fiable
3.5 Transporte orientado a conexión: TCP
- 3.5.1 La conexión TCP
- 3.5.2 Estructura del segmento TCP
- 3.5.3 Estimación del tiempo de ida y vuelta y fin de temporización
- 3.5.4 Transferencia de datos fiable
- 3.5.5 Control de flujo
- 3.5.6 Gestión de la conexión TCP

3.6 Principios de control de congestión
3.7 Mecanismo de control de congestión de TCP

TCP: Control de flujo

- Procedimiento TCP para “frenar”– regular la velocidad de transmisión de datos.
  - Servicio que permite adaptar la velocidad entre Emisor y Receptor.
- Objetivo ➔ No desbordar el *buffer* del Receptor TCP.
  - Por cuanto el proceso de Aplicación en el receptor NO lee los datos del *buffer* en el instante de llegada, hay cierta demora …mayor o menor…
- Importante.
  - Control de flujo NO ES LO MISMO que control de la congestión, si bien ambos conllevan a “frenar” al Emisor.

TCP: Procedimiento TCP para control de flujo (I)

- Suponga:
  - Que se descartan de segmentos fuera de secuencia.
  - Que el receptor gestiona un "buffer de recepción" ➔ Buffer del Receptor.
  - Emisor conoce el valor de la variable "ventana de recepción".
- ¿Qué indica la variable "Ventana de recepción"?
  - Proporciona información del espacio libre en el *buffer* del Receptor.
    - Tamaño del *buffer* = BufferRecepcion
    - Variables asociadas al *buffer*:
      - UltimoByteLeido
      - UltimoByteRecibido

TCP: Procedimiento TCP para control de flujo (II)

- $\text{ÚltimoByte}_{\text{Recibido}} - \text{ÚltimoByte}_{\text{Leído}} \leq \text{Buffer}_{\text{Recepción}}$
- $\text{Ventana}_{\text{Recepción}} = \text{Buffer}_{\text{Recepción}} - [\text{ÚltimoByte}_{\text{Recibido}} - \text{ÚltimoByte}_{\text{Leido}}]$
  - $\text{Ventana}_{\text{Recepción}}$ ← Es una Variable dinámica que varía en el tiempo.

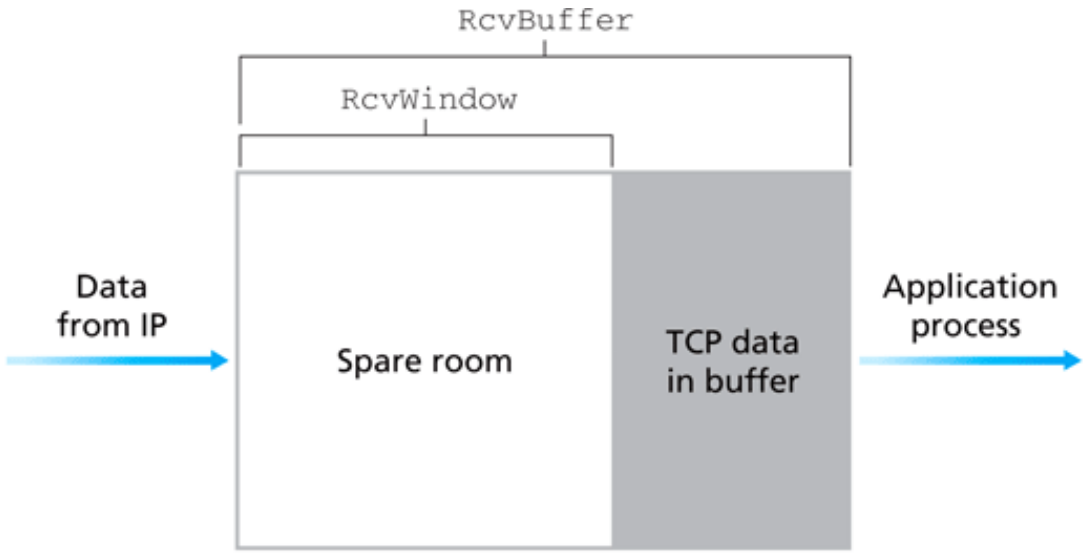


Figure 3.38 ♦ The receive window (RcvWindow) and the receive buffer (RcvBuffer)

TCP: Procedimiento TCP para control de flujo (III)

- Variable "$Ventana_{Recepción}$" y Control de Flujo
  - El receptor notifica el espacio libre de su *buffer* mediante el campo "ventana de recepción" en cada segmento que transmite.
  - Receptor inicialmente establece: $Ventana_{Recepción} = Buffer_{Recepción}$
  - Emisor
    - Controla las variables "$UltimoByte_{Enviado}$" y "$UltimoByte_{Confirmado}$".
      - $UltimoByte_{Enviado} - UltimoByte_{confirmado} = Cantidad\ de\ datos\ NO\ reconocidos$.
    - Garantiza que, durante todo el tiempo de vida de la conexión TCP:
      - $\underbrace{UltimoByte_{Enviado} - UltimoByte_{confirmado}}_{\text{Cantidad de datos no reconocidos}} \leq Ventana_{Recepción}$

TCP: Procedimiento TCP para control de flujo (IV): Problema

- Si Receptor:
  - *Tiene el Buffer* lleno ➔ $Ventana_{Recepcion}$ = 0 ➔ Se notifica al Emisor.
  - NADA que enviar ➔ El proceso de Aplicación saca los datos del *buffer*, pero NO envía ningún segmento al Emisor (ni siquiera ACKs).
    - NO se envían segmentos con nuevos valores para la "$Ventana_{Recepcion}$".
    - El emisor NO es informado de la disponibilidad de espacio en el *buffer* del Receptor ➔ Emisor bloqueado, NO envía datos.
- SOLUCIÓN.
  - Cuando el Emisor conoce que el Receptor tiene lleno el *buffer* (**$Ventana_{Recepcion}$ = 0**):
    - Continua enviando segmentos con sólo un octeto de datos cada 60 seg.
    - Estos segmentos serán confirmados por el Receptor ➔ Emisor será notificado cuando el Receptor disponga de espacio libre en su buffer ➔ Emisor NO queda bloqueado.

Tema 3: La capa de Transporte

3.1 La capa de Transporte y sus servicios
3.2 Multiplexacción y demultiplexación
3.3 Transporte sin conexión: UDP
3.4 Principios de transferencia de datos fiable
3.5 Transporte orientado a conexión: TCP
- 3.5.1 La conexión TCP
- 3.5.2 Estructura del segmento TCP
- 3.5.3 Estimación del tiempo de ida y vuelta y fin de temporización
- 3.5.4 Transferencia de datos fiable
- 3.5.5 Control de flujo
- 3.5.6 Gestión de la conexión TCP

3.6 Principios de control de congestión
3.7 Mecanismo de control de congestión de TCP

TCP: Gestión de la conexión

- Control del establecimiento y terminación de la conexión TCP.
  - Tras este procedimiento subyacen dos cosas muy importantes:
    - Puede aumentar SIGNIFICATIVAMENTE el retardo entre sistemas finales.
    - Muchos de los ataques de red más comunes explotan las vulnerabilidades intrínsecas de la gestión de conexión TCP.
  - Establecimiento de la conexión TCP.
    - Tres pasos, tres fases ➔ Tres segmentos especiales (de control), acuerdo en tres fases.
    - Acuerdo/negociación de la conexión, en base a los contenidos de las cabeceras TCP obligatorias y opcionales, contenidas en los paquetes de control iniciales.
  - Cierre de la conexión TCP.
    - Cuatro pasos, cuatro fases ➔ Cuatro segmentos especiales (de control).

TCP: Pasos para establecer una conexión TCP (I)

- Paso 1.
  - El Cliente.
    - Selecciona de manera aleatoria el Nº de secuencia inicial ($cliente_{nsi}$).
      - Requiere procedimiento apropiado para evitar ciertos ataques de seguridad.
    - Envía segmento SYN.
      - Campo Nº de secuencia = $cliente_{nsi}$.
      - Indicador SYN activado ➔ SYN = 1, información de control.
      - Indicador ACK desactivado ➔ ACK = 0
      - NO contiene datos.

TCP: Pasos para establecer una conexión TCP (II)

- Paso 2
  - El Servidor.
    - Recibe el segmento SYN.
    - Asigna *buffer* y variables a la conexión.
      - ➔ Vulnerabilidad (ver páginas 254 – 255 del libro de texto).
        - Ataques por «inundación» de SYN.
    - Responde, enviando un segmento de “conexión aceptada” (Segmento SYN-ACK).
      - NO contiene datos.
      - Contiene información de control.
        - Campo “Nº de secuencia” = $servidor_{nsi}$ (seleccionado de manera aleatoria).
        - Campo “Nº de reconocimiento” = $cliente_{nsi}$ + 1
        - Indicador SYN activado ➔ SYN = 1
        - Indicador ACK activado ➔ ACK = 1

TCP: Pasos para establecer una conexión TCP (III)

- Paso 3
  - El Cliente.
    - Recibe segmento SYN-ACK.
    - Asigna *buffer* y variables a la conexión.
    - Envía segmento "reconocimiento (ACK) de conexión aceptada".
      - Contiene información de control.
        - Campo "Nº de reconocimiento" = $servidor_{nsi}$ + 1.
        - Indicador SYN desactivado ➔ SYN = 0
        - Indicador ACK activado ➔ ACK = 1
      - Puede o no contener datos.

TCP: Pasos para establecer una conexión TCP (IV)

- Acuerdo en tres fases.
  - Para "sincronizar" a las partes.
- Parte que inicia la conexión actúa como Cliente.
- Una vez establecida la conexión las partes pueden intercambiar datos mediante "segmentos de datos".
  - Indicador SYN = 0.

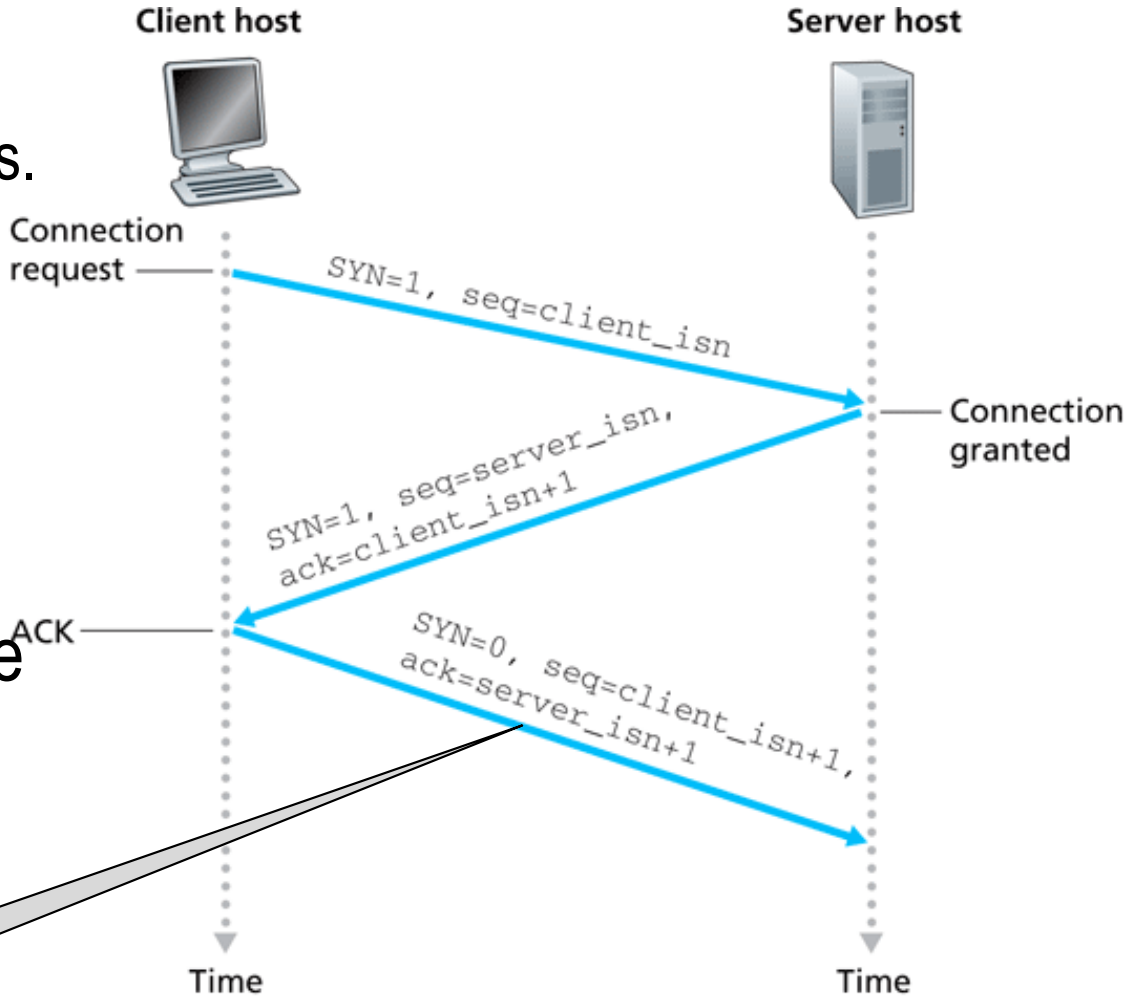


Figure 3.39 ♦ TCP three-way handshake: segment exchange

TCP: Pasos para cerrar una conexión TCP (I)

- Paso 1.
  - Parte que inicia el proceso de cierre de la conexión:
    - Envía segmento FIN ➔ Indicador FIN activado, FIN = 1
- Paso 2.
  - La otra Parte:
    - Envía (devuelve) segmento ACK.
- Paso 3.
  - La otra parte, cuando proceda:
    - Envía segmento FIN ➔ Indicador FIN activado, FIN = 1.
- Paso 4.
  - Parte que inició el proceso de cierre de la conexión.
    - Envía (devuelve) segmento ACK en respuesta al segmento FIN recibido.

TCP: Pasos para cerrar una conexión TCP (II)

- Cualquiera de las partes puede cerrar la conexión.
- Conexión terminada:
  - Se liberan los recursos utilizados.
    - *Buffer* y variables.
    - Nºs de puerto.

Tiempo de espera por si hay que re-enviar el ACK. ***Timeout*** dependiente de la Aplicación. Valores típicos: 30, 60 y **120** segundos

Toda una "eternidad" en las redes actuales. Establecido en el estándar inicial.

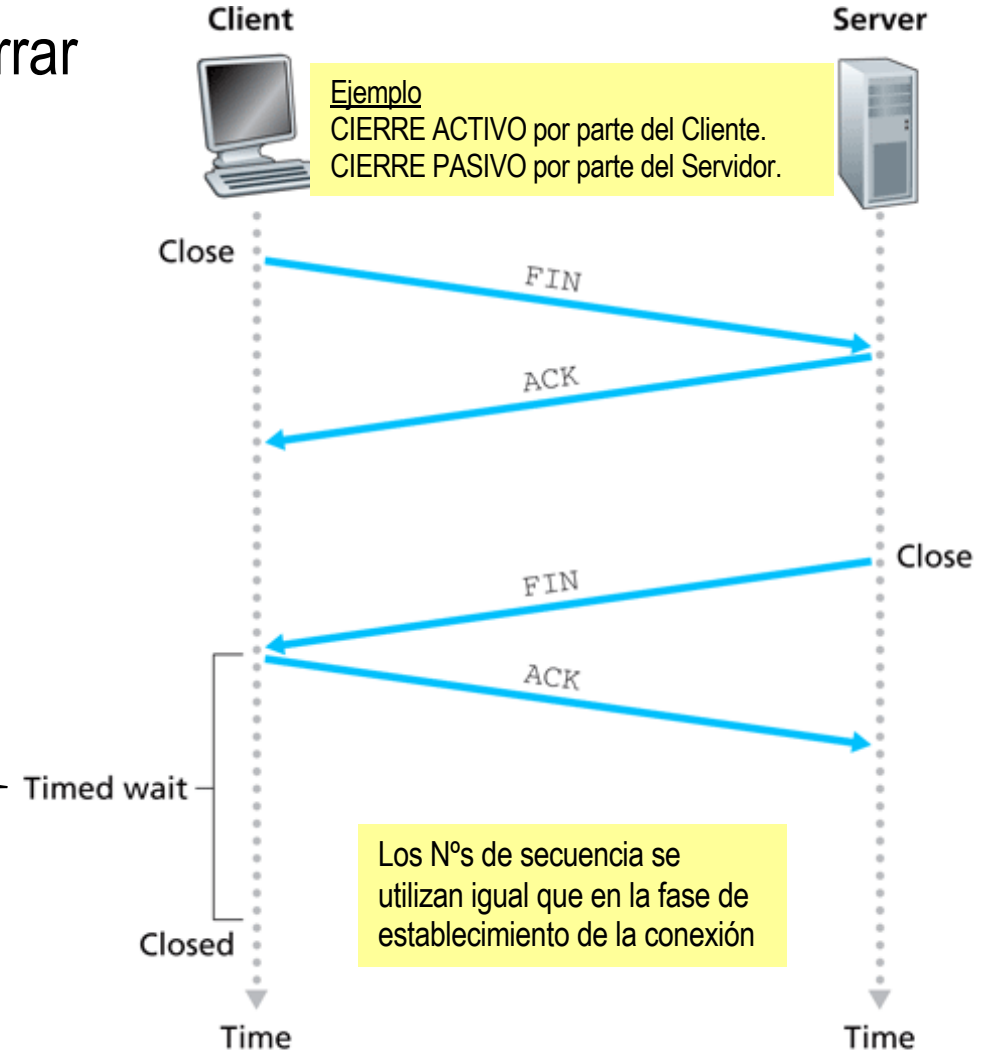


Figure 3.40 ♦ Closing a TCP connection

TCP: Estados TCP en el lado Cliente

Nota: Lado Cliente inicia el cierre de la conexión

Este diagrama sólo muestra el establecimiento y cierre **normal** de una conexión TCP.
Otros escenarios más complejos se pueden presentar en la práctica. Ver fuente sugerida en el libro de texto (página 252).

Este diagrama corresponde con la **figura 3.40** del libro de texto, donde se muestran explícitamente los cuatro pasos para el cierre de la conexión TCP

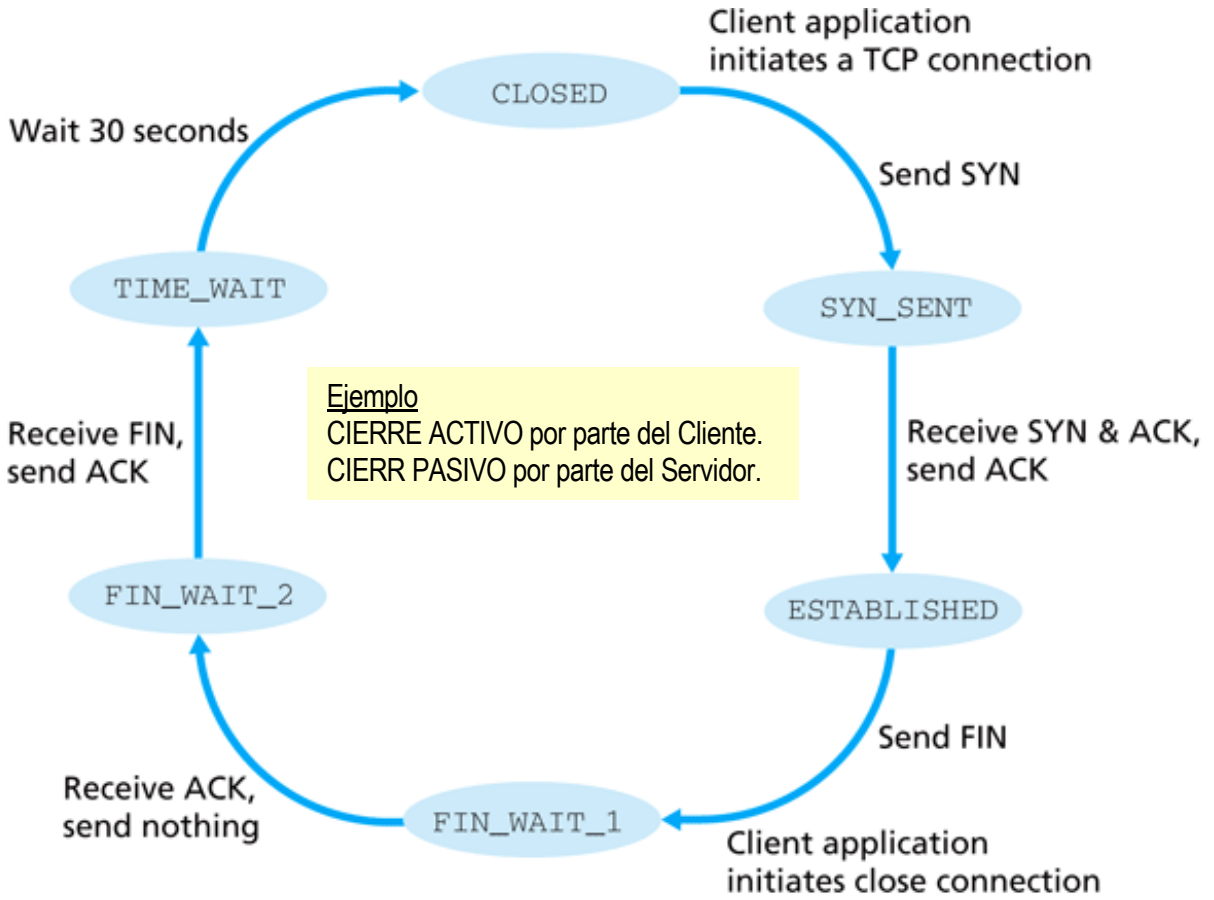


Figure 3.41 ♦ A typical sequence of TCP states visited by a client TCP

TCP: Estados TCP en el lado Servidor

Nota: Lado Cliente inicia el cierre de la conexión

Este diagrama sólo muestra el establecimiento y cierre **normal** de una conexión TCP.
Otros escenarios más complejos se pueden presentar en la práctica. Ver fuente sugerida en el libro de texto (página 252).

Este diagrama corresponde con la **figura 3.40** del libro de texto, donde se muestran explícitamente los cuatro pasos para el cierre de la conexión TCP

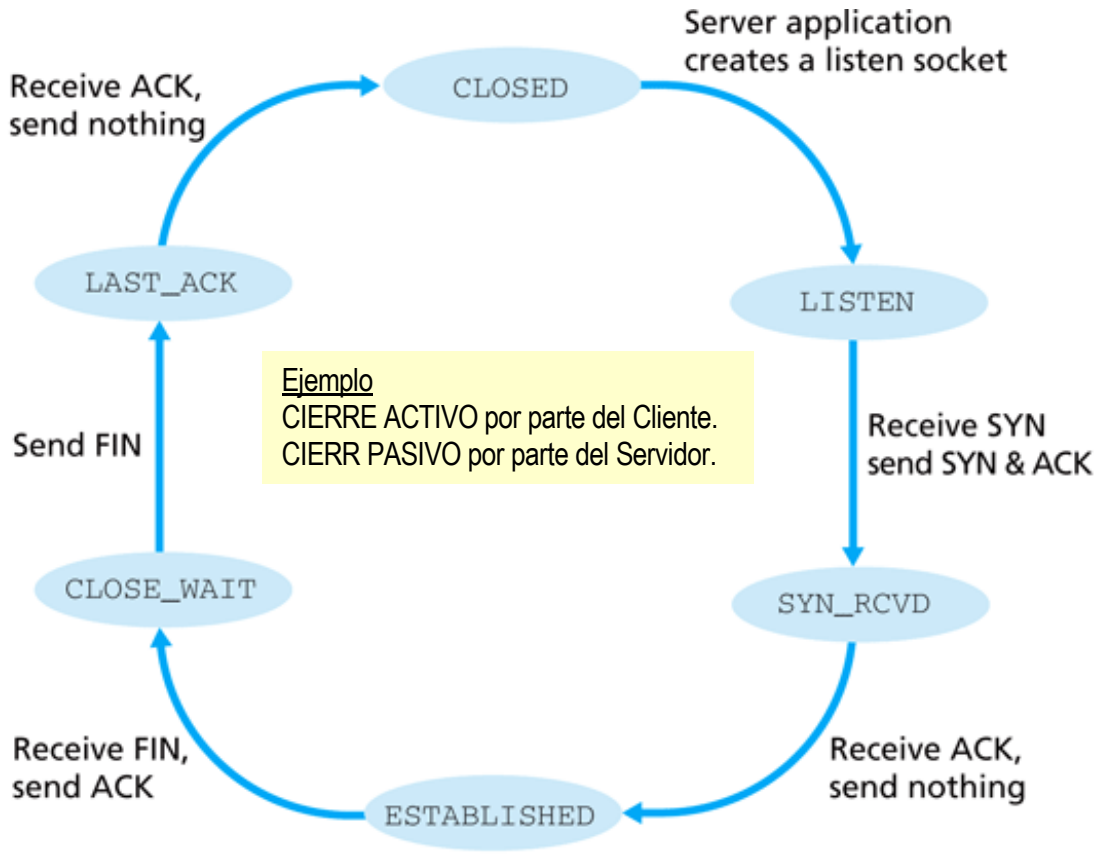


Figure 3.42 ♦ A typical sequence of TCP states visited by a server-side TCP